Reducing the Footprint of Multi-Vector Retrieval with Minimal Performance Impact via Token Pooling

Benjamin Clavié
bc@answer.ai
Answer.AI
Japan

Antoine Chaffin
antoine.chaffin@lighton.ai
LightOn
France

Griffin Adams
ga@answer.ai
Answer.AI
USA

arXiv:2409.14683v1 [cs.IR] 23 Sep 2024

ABSTRACT

Over the last few years, multi-vector retrieval methods, spearheaded by ColBERT, have become an increasingly popular approach to Neural IR. By storing representations at the token level rather than at the document level, these methods have demonstrated very strong retrieval performance, especially in out-of-domain settings. However, the storage and memory requirements necessary to store the large number of associated vectors remain an important drawback, hindering practical adoption. In this paper, we introduce a simple clustering-based token pooling approach to aggressively reduce the number of vectors that need to be stored. This method can reduce the space & memory footprint of ColBERT indexes by 50% with virtually no retrieval performance degradation. This method also allows for further reductions, reducing the vector count by 66%-to-75% , with degradation remaining below 5% on a vast majority of datasets. Importantly, this approach requires no architectural change nor query-time processing, and can be used as a simple drop-in during indexation with any ColBERT-like model.

CCS CONCEPTS

• **Information systems → Retrieval models and ranking**.

KEYWORDS

Neural IR, Late Interaction, ColBERT, Multi-Vector Retrieval, Token Pruning, Pooling, Index Compression

ACM Reference Format:
Benjamin Clavié, Antoine Chaffin, and Griffin Adams. 2024. Reducing the Footprint of Multi-Vector Retrieval with Minimal Performance Impact via Token Pooling. In *Proceedings of (ArXiv Preprint).* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Deep-learning based approaches are becoming increasingly popular in retrieval. Effectively, this means using a neural network to assess the relevance of documents for a given query. Many different approaches to neural IR exist, such as single-vector dense representations [32], learned sparse representations [8], or late-interaction methods, such as ColBERT [11], which employ multi-vector representations by storing a vector for each token in a document.

Using multiple vectors to represent the sequences allows for a more fine-grained and expressive representation compared to compressing all the semantics into a single vector. As a result, this latter approach has frequently been shown to generalize better to out-of-domain settings than dense representations [16, 34]. These properties have led to these methods experiencing a considerable amount of interest in the recent months[1].

However, the enhanced expressiveness of multi-vector representations comes at a hefty storage and memory cost. It is relatively trivial to store millions of dense representations using common indexing methods such as HNSW [20]. On the other hand, ColBERT and similar approaches require an order of magnitude more vectors per document. Many techniques have been proposed to alleviate this issue: dimension reduction methods through a learned projection layer [11], aggressive quantization using Inverted File [36]-Product Quantization [10] (IVF-PQ) indexing [27] and an optimised indexing and querying mechanism [26].

These methods, while bringing the storage requirement down to the same order of magnitude as single-vector methods, does not fully close the gap with storage and memory requirements of these approaches. This is especially apparent as dense single-vector representations can also be aggressively quantized [33], with relatively little performance loss when used in conjunction with quantization-aware training [24].

Moreover, the indexing methods required to achieve this level of compression with ColBERT add complex processing layers, constraining the index to a specific format. This renders them considerably less flexible, notably in terms of document addition and deletion (*CRUD*) than commonly used dense indexing methods such as HNSW [20]. As a result, ColBERT can appear as a worse option for evolving corpora, limiting its practical use.

Another line of work has focused on attempting to reduce the total number of vectors needed to be stored. However, the existing methods have not reached widespread adoption, either due to requiring a modified pipeline and specific training for limited gains [9]; or as a result of minimal storage reduction in comparison to the negative retrieval performance impact [14].

Recently, a study has also confirmed the intuition that different tokens have vastly varying levels of importance for multi-vector retrieval performance [15]. This is in line with a series of studies in the adjacent field of Computer Vision, which have introduced *token*

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ArXiv Preprint, June 03–05, 2018, Woodstock, NY
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

[1]Indeed, ColBERTv2 has gone from 40,000 monthly downloads in late 2023 to being one of the most downloaded models on the HuggingFace model hub, with over 5 million monthly downloads. See https://huggingface.co/colbert-ir/colbertv2.0

merging [2], which consists in averaging the representation of multiple tokens into a single vector. These studies have shown that it is possible to merge tokens without requiring specific training, greatly improving the throughput of image processing [2] or image generation [3], with very minimal impact on task-specific performance.

Contributions In this work, ① we introduce a method we call Token Pooling, which requires no specific training nor pipeline or model modification. This method consists in reducing the total number of vectors retained at indexing time, by average pooling token representations through simple clustering approaches. We explore different clustering methods and show that hierarchical clustering yields the best results. On a variety of commonly used evaluation datasets, this method performs very strongly, allowing for storage cost reductions of up to **50%** with no performance degradation on average, and storage cost reductions of **66%** with fewer than **3%** degradation. ② We then demonstrate that this approach can be coupled with the usual ColBERT quantization pipeline described above, achieving even greater compression results. ③ Finally, we show that the approach also applies to more than just the ColBERTv2 model and the English language, by showing strong results with Japanese variant of ColBERT on Japanese data [6].

2 TOKEN POOLING

To try and mitigate the issues associated with the high number of vectors to store when using multi-vector retrieval methods, we introduce a simple **Token Pooling** approach. This is applied at indexing time to reduce the effective number of tokens representing a document. It can be used with any pre-trained ColBERT model, without any further training or modifications.

Our core approach is simple and works as a two-step system: First, we devise a way to group individual vectors together, using one of the three clustering methods detailed below. We then apply **mean pooling** in order to obtain a single vector which contains an average representation of the cluster. The resulting set of pooled vectors serves as our new multi-vector document representation, and the original vectors are discarded. The main intuition behind this approach is the belief that there is considerable redundancy token vectors within as single a document, and that tokens therefore have varying importance [15]. Within this assumption, pooling the most similar ones is unlikely to considerably modify the overall document representation.

To control the level of compression, we introduce a new variable called the pooling factor. This factor is, effectively, a compression factor. For example, a pooling factor of 2 reduces the total number of vectors stored by a factor of 2, i.e. a 50% reduction.

2.1 Pooling Methods

We explore three pooling methods:

Sequential Pooling. This method acts as our baseline does not require any clustering. Tokens are pooled together based on the order in which they appear in the document, from left to right. In this setting, the pooling factor dictates the number of sequential tokens pooled together. We do not use a sliding window, which means that each token is only ever pooled once. This baseline is inspired by the common intuition that the individual meaning of words is greatly influenced by its direct neighbours [5, 7].

K-Means based pooling. This method uses k-means clustering [18] based on the cosine distance between vectors. The pooling factor is used in this setting to define the total number of clusters, which is set at initial token count/Pooling Factor + 1.

Hierarchical clustering based pooling. This method uses hierarchical clustering [21], again based on the cosine distance between vectors. We use Ward's method [31] to produce our clusters, which intuitively would be well-suited for this task, as it would seek to minimize the distance between the original vector and the pooled outputs. Additionally, it has generally been observed to perform well for text data [12, 25].

Using this method, we effectively iteratively merge the vectors that minimize the ward distance. In this setting, the pooling factor is used to define the maximum number of clusters that can be formed, as a constraint on the clustering algorithm. This effectively means that this method will result in **at most** initial token count/Pooling Factor + 1 clusters.

3 EXPERIMENTAL SETTING

3.1 Implementation

Models We conduct the vast majority of our experiments with ColBERTv2 [27], trained on English MS-Marco [22]. To assess that our method is not specific to English nor to ColBERTv2, we also conduct a smaller set of experiments on Japanese using a Japanese version of ColBERT, JaColBERTv2 [6].

Clustering All clustering methods are implemented using existing libraries and widely used implementations. We use SciPy [30] for hierarchical clustering and a simple, PyTorch-based [23] implementation for k-means clustering. We evaluate clustering on a wide range of pooling factors: 2, 3, 4, 5, 6 and 8.

Indexing All experiments are conducted using the official ColBERT implementation to encode both queries and documents. 2-bit quantization and PLAID [26] indexing are also performed with the original codebase. Experiments with non-quantized vectors are conducted using a basic HNSW indexing implementation, via the voyager library. We provide further details on indexing parameters in Appendix A.

3.2 Evaluation

Data We evaluate our method on a varied mix of datasets, in order to capture the impact of the approach in different domains. To do so, we select all small to mid-sized datasets from BEIR [29], the most commonly used retrieval evaluation suite, with the exception of ArguAna[2], and from LoTTe [27], another benchmark commonly used for multi-vector approaches. We define as "small and mid-sized" any dataset containing fewer than 500,000 documents. We decide on this cutoff in order to appropriately explore a variety of data sizes while keeping down the cost of experimentation and analysis. In total, this results in 6 datasets from BEIR and 3 from

[2]Due to sensibly diverging results between our ArguAna baseline reproduction and reported results in the literature, as well as the less representative nature of ArguAna, as it is an argument mining dataset, we have not included it in this study

	BEIR				
	scifact	scidocs	nfc	fiqa	**Avg**
Hierarc.					
pool 2	**100.22**	97.90	**102.02**	**102.33**	**100.62**
pool 3	99.99	97.00	100.71	98.41	99.03
pool 4	96.93	95.16	101.10	94.91	97.03
pool 6	92.44	87.48	96.58	86.17	90.67
KMeans					
pool 2	95.09	99.75	97.35	97.31	97.38
pool 3	93.13	99.65	96.16	94.90	95.96
pool 4	91.62	98.17	94.36	90.24	93.60
pool 6	89.59	94.62	91.63	84.82	90.166
Seq.					
pool 2	95.37	**103.01**	96.46	95.41	97.56
pool 4	87.80	95.91	85.78	82.61	88.03

Table 1: Relative performance of various token pooling methods, applied to 16-bit unquantized vectors with HNSW Indexing. A score of 100 corresponds to the model performance without any pooling, and all results are relative to it.

LoTTe. For LoTTe datasets, we use the *search* query subset. We evaluate token pooling with uncompressed vectors on only datasets with fewer than 100,000 documents, and compressed 2-bit vectors on all our chosen datasets.

For the evaluation of JaColBERTv2, we use two commonly used datasets for Japanese retrieval evaluations, with the same evaluation setting as in the original JaColBERT report [6]: JSQuAD [13] and the Japanese split of MIRACL [35].

Metrics We report all results in terms of relative performance, where 100 represents the score obtained by the same model with no token pooling. For all datasets, the metrics used to compute this relative performance are based on the ones previously used in the literature. Respectively, NDCG@10 for all BEIR datasets, Success@5 for LoTTe datasets and Recall@5 for JSQuAD and MIRACL (Japanese split). All metrics are computed using the RANX [1] library.

4 RESULTS

4.1 Unquantized Results

Table 1 presents the detailed results for various pooling factors and pooling methods. In the interest of simplicity and space, we only report the results of sequential pooling for factors 2 and 4, as the observed performance degradation varies wildly between datasets and increases is too quickly for this approach to be viable in comparison to the other ones. Worth noting however that, despite its overall noticeably weaker performance, sequential pooling performs remarkably strongly on the SCIDOCS dataset, reaching the strongest performance at a pooling factor of 2, before degrading behind the other two methods at factor 4. An overview of the relative performance degradation of the best-performing pooling method, hierarchical pooling, across pooling factors on all the small-sized evaluation datasets can be found in Figure 1 .

We observe that token pooling performs remarkably well in this setting, on all four datasets. In fact, a pooling factor of 2, resulting in a vector count reduction of 50%, actually slightly **increases** retrieval performance on average, doing so on 3 out of 4 datasets. A pooling factor of 3 achieves an average performance degradation of less than 1%, despite reducing storage requirements by over 66%. We observe a starker degradation from pooling factors of 4 onwards, with an average degradation of 3% and a degradation of around 5% on two of the evaluated datasets.

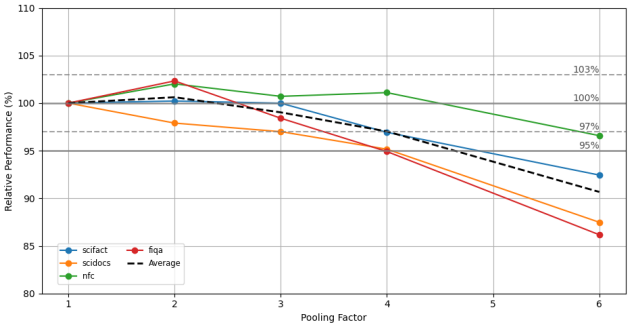

Figure 1: Relative performance degradation at various pooling factors using 16-bit vectors with HNSW indexing.

Further compression continues to degrade performance, with a prohibitive average degradation of 10% at a pooling factor of 6. However, the very strong results of the lower pooling factors show that it is possible to drastically reduce the number of vectors needing to be stored for late-interaction retrieval, at virtually no cost to retrieval performance. These results are especially promising as the lower pool factors offer a comparatively greater gain than subsequent increases.

4.2 Quantized Results

The next experiment focuses on evaluating whether this method can be combined with standard ColBERTv2 quantization, enabling its practical use on larger datasets.

As above, we report the detailed results in Table 2, with the full sequential results truncated, and the overview of relative performance degradation with hierarchical pooling in Figure 2.

Immediately, we notice one obvious outlier: Touché [4]. Not only does performance never decrease with pooling, but it steadily increases, up to a 42.16% increase in retrieval performance at a pooling factor of 6. This dataset has frequently been noted as producing unusual results and a recent study has shown that due to its nature as an argument-mining dataset repurposed for retrieval, it is an outlier in many ways and contains considerable noise [28]. As a result, we choose to leave further exploration of this behavior for future work.

There is a second, less-pronounced, outlier in place of fiqa. However, unlike Touché, it does follow a similar trend to other datasets, although the performance degrades noticeably faster as the pool factor increases. While we leave further analysis to future work, fiqa is a highly specialized dataset within the financial domain, and its queries tend to focus on very fine-grained details [19], which could partially explain this quicker degradation.

	BEIR								LOTTE				Overall
	scifact	scidocs	nfc	fiqa	trec covid	Touché	**Avg**	**w/o outliers**	Wri-ting	Recrea.	Life-style	**Avg**	**Avg w/o outliers**
Hierarc.													
pool 2	**98.11**	98.76	**98.87**	**98.16**	102.95	112.50	101.56	**99.67**	**99.65**	**100.61**	98.54	**99.60**	**99.64**
pool 3	94.12	98.74	94.15	89.80	101.27	123.29	100.23	97.07	97.78	96.07	97.62	97.16	97.11
pool 4	93.28	98.10	93.22	84.46	101.05	132.34	100.41	96.41	96.73	93.04	96.70	95.49	96.02
pool 6	90.23	94.34	91.23	74.63	101.45	136.49	98.06	94.31	92.99	84.72	91.58	89.76	92.36
KMeans													
pool 2	94.18	**100.05**	93.25	89.18	**103.11**	131.23	**101.83**	97.65	97.31	96.97	**98.72**	97.67	97.66
pool 3	93.71	98.76	89.57	83.23	101.75	136.24	100.54	95.95	96.15	94.10	96.34	95.53	95.77
pool 4	94.41	97.77	87.89	80.54	99.66	138.94	99.87	94.93	94.39	90.92	93.95	93.09	94.14
pool 6	87.35	90.10	85.80	72.28	96.46	**142.13**	95.69	89.93	90.42	81.09	90.51	87.34	88.82
Seq.													
pool 2	96.55	99.34	94.07	89.75	94.67	84.16	93.09	96.16	88.90	93.04	95.79	92.58	94.62
pool 4	86.47	89.29	78.44	74.13	78.48	91.20	83.01	83.17	80.49	80.79	90.66	83.98	83.52

Table 2: Relative performance of various token pooling methods, applied to 2-bit quantized vector with PLAID Indexing. A score of 100 corresponds to the model performance without any pooling, and all results are relative to it.

Discounting outliers, we observe overall similar results in the quantized settings as with unquantized vectors. Performance degradation appears to be slightly more pronounced but still very contained, with an average degradation of 1.34% at a pool factor of 2, and 3.52% at a pool factor of 3. While nearly all datasets share the same overall trend, there are noticeable variations between datasets, with some beginning to decrease more rapidly at higher pool factors, while others retain performance, even with aggressive pooling.

Interestingly, we note that similarly to the 16-bit setting, KMeans clustering outperforms hierarchical clustering on the SCIDOCS dataset at a pool factor 2, and observe a similar behaviour. However, the performance of hierarchical clustering still noticeably outperforms it on average at every pool factor, and widens the gap at higher pool factors.

4.3 Vector Count & Storage Reduction

Table 3 provides a comparison in the number of vector stored as well as the disk footprint of storing the full index. We use TREC-Covid at a truncated length of 256 tokens per document (ColBERTv2's default document length [27]) for these calculations. We report the number of vectors and index size for 16-bit single-vector dense representations in an HNSW index, as well as PLAID-indexed ColBERT at various pooling factors. Note that the index size reduction is slightly lower than the vector count reduction, due to the indexing overhead introduced by PLAID [26].

We choose to report 16-bit for dense vectors and 2-bit for PLAID, as both methods experience virtually no performance degradation, therefore comparing index without quantization-related performance compromises.

4.4 Japanese Results

Finally, Table 4 shows the results of token pooling with hierarchical clustering applied to Japanese corpora, using the JaColBERTv2 [6] model. This evaluation is performed only in the quantized setting, with 2-bit compression on a PLAID [26] index.

	Index Size
16-bit Dense Vector	345 MB
2-bit PLAID Index	760 MB
pool 2	388 MB
pool 3	260 MB
pool 4	195 MB
pool 6	131 MB

Table 3: An overview of index size (in Megabytes) between single-vector representation and PLAID ColBERT indexes.

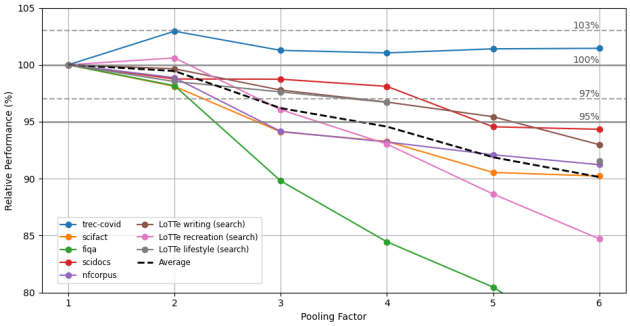

Figure 2: Relative performance degradation at various pooling factors using PLAID-indexed 2-bit vectors. fiqa at factor 6 is truncated for readability.

While not as thoroughly explored as the English ones, these results show that the pooling approach is neither unique to the ColBERTv2 model nor the English language, as we observe a similar pattern. Performance degradation is minimal at lower pooling factors of 2 and 3, despite the impressive compression.

	JSQuAD	MIRACL	Avg
Hierarc.			
pool 2	99.68	100.03	99.855
pool 3	98.62	97.75	98.185
pool 4	97.69	98.26	97.975
pool 6	96.22	93.97	95.095

Table 4: Relative performance of hierarchical clustering-based pooling on two Japanese datasets, using quantized 2-bit vectors and PLAID indexing.

5 CONCLUSION

In this paper, we introduced TOKEN POOLING, a simple approach leveraging existing clustering methods and requiring no training nor model modifications to very effectively reduce the numbers of tokens needed to store for multi-vector retrieval models such as ColBERT. Our results show that the number of vectors, can be reduced by 50% with little-to-no performance degradation on the majority of datasets evaluated, thus drastically reducing the size of ColBERT indexes. Reducing the vector count by 66% still exhibits minimal degradation, while further compression results in increasing performance degradation. Notably, this method can lead to ColBERT being applied to a broader range of uses, as it facilitates the use of addition/deletion (*CRUD*)-friendly indexing methods such as HNSW. Our experiments also show that these results hold true even when combined with ColBERT's 2-bit quantization process, allowing for even greater compression than previously possible. We also reproduce our results using a Japanese ColBERT model, showing that this is not limited to a single model, nor to English documents. With this paper focusing on introducing this method and demonstrating its empirical performance, we hope that our findings will help support future research in better understanding the role of individual tokens in multi-vector retrieval and develop even stronger compression methods.

ACKNOWLEDGMENTS

We would like to thank Omar Khattab for his valuable and enthusiastic encouragements while working on this paper.

REFERENCES

[1] Elias Bassani. 2022. ranx: A blazing-fast python library for ranking evaluation and comparison. In *European Conference on Information Retrieval*. Springer, 259–264.

[2] Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. 2022. Token merging: Your vit but faster. *arXiv preprint arXiv:2210.09461* (2022).

[3] Daniel Bolya and Judy Hoffman. 2023. Token merging for fast stable diffusion. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 4599–4603.

[4] Alexander Bondarenko, Maik Fröbe, Meriem Beloucif, Lukas Gienapp, Yamen Ajjour, Alexander Panchenko, Chris Biemann, Benno Stein, Henning Wachsmuth, Martin Potthast, et al. 2020. Overview of Touché 2020: argument retrieval. In *Experimental IR Meets Multilinguality, Multimodality, and Interaction: 11th International Conference of the CLEF Association, CLEF 2020, Thessaloniki, Greece, September 22–25, 2020, Proceedings 11*. Springer, 384–395.

[5] Mikael Brunila and Jack LaViolette. 2022. What company do words keep? Revisiting the distributional semantics of JR Firth & Zellig Harris. *arXiv preprint arXiv:2205.07750* (2022).

[6] Benjamin Clavié. 2023. Jacolbert and hard negatives, towards better japanese-first embeddings for retrieval: Early technical report. *arXiv preprint arXiv:2312.16144* (2023).

[7] John Firth. 1957. A synopsis of linguistic theory, 1930-1955. *Studies in linguistic analysis* (1957), 10–32.

[8] Thibault Formal, Benjamin Piwowarski, and Stéphane Clinchant. 2021. SPLADE: Sparse lexical and expansion model for first stage ranking. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2288–2292.

[9] Sebastian Hofstätter, Omar Khattab, Sophia Althammer, Mete Sertkan, and Allan Hanbury. 2022. Introducing neural bag of whole-words with colberter: Contextualized late interactions using enhanced reduction. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*. 737–747.

[10] Herve Jegou, Matthijs Douze, and Cordelia Schmid. 2010. Product quantization for nearest neighbor search. *IEEE transactions on pattern analysis and machine intelligence* 33, 1 (2010), 117–128.

[11] Omar Khattab and Matei Zaharia. 2020. Colbert: Efficient and effective passage search via contextualized late interaction over bert. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*. 39–48.

[12] Tuomo Korenius, Jorma Laurikkala, Martti Juhola, and Kalervo Järvelin. 2006. Hierarchical clustering of a Finnish newspaper article collection with graded relevance assessments. *Information Retrieval* 9 (2006), 33–53.

[13] Kentaro Kurihara, Daisuke Kawahara, and Tomohide Shibata. 2022. JGLUE: Japanese general language understanding evaluation. In *Proceedings of the Thirteenth Language Resources and Evaluation Conference*. 2957–2966.

[14] Carlos Lassance, Maroua Maachou, Joohee Park, and Stéphane Clinchant. 2022. Learned token pruning in contextualized late interaction over BERT (colbert). In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2232–2236.

[15] Jinhyuk Lee, Zhuyun Dai, Sai Meher Karthik Duddu, Tao Lei, Iftekhar Naim, Ming-Wei Chang, and Vincent Zhao. 2024. Rethinking the role of token retrieval in multi-vector retrieval. *Advances in Neural Information Processing Systems* 36 (2024).

[16] Simon Lupart, Thibault Formal, and Stéphane Clinchant. 2023. Ms-shift: An analysis of ms marco distribution shifts on neural retrieval. In *European Conference on Information Retrieval*. Springer, 636–652.

[17] Sean MacAvaney and Nicola Tonellotto. 2024. A Reproducibility Study of PLAID. *arXiv preprint arXiv:2404.14989* (2024).

[18] James MacQueen et al. 1967. Some methods for classification and analysis of multivariate observations. In *Proceedings of the fifth Berkeley symposium on mathematical statistics and probability*, Vol. 1. Oakland, CA, USA, 281–297.

[19] Macedo Maia, Siegfried Handschuh, André Freitas, Brian Davis, Ross McDermott, Manel Zarrouk, and Alexandra Balahur. 2018. Www'18 open challenge: financial opinion mining and question answering. In *Companion proceedings of the the web conference 2018*. 1941–1942.

[20] Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence* 42, 4 (2018), 824–836.

[21] Fionn Murtagh and Pedro Contreras. 2012. Algorithms for hierarchical clustering: an overview. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* 2, 1 (2012), 86–97.

[22] Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and Li Deng. 2016. Ms marco: A human-generated machine reading comprehension dataset. (2016).

[23] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).

[24] Neils Reimers. 2024. *Cohere int8 & binary Embeddings - Scale Your Vector Database to Large Datasets*. Technical Report. https://cohere.com/blog/int8-binary-embeddings

[25] Niyaz M Salih and Karwan Jacksi. 2020. Semantic Document Clustering using K-means algorithm and Ward's Method. In *2020 International Conference on Advanced Science and Engineering (ICOASE)*. IEEE, 1–6.

[26] Keshav Santhanam, Omar Khattab, Christopher Potts, and Matei Zaharia. 2022. PLAID: an efficient engine for late interaction retrieval. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*. 1747–1756.

[27] Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. 2021. Colbertv2: Effective and efficient retrieval via lightweight late interaction. *arXiv preprint arXiv:2112.01488* (2021).

[28] Nandan Thakur, Luiz Bonifacio, Maik Fröbe, Alexander Bondarenko, Ehsan Kamalloo, Martin Potthast, Matthias Hagen, and Jimmy Lin. 2024. Systematic Evaluation of Neural Retrieval Models on the Touché 2020 Argument Retrieval Subset of BEIR. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*.

[29] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. Beir: A heterogenous benchmark for zero-shot evaluation of information retrieval models. *arXiv preprint arXiv:2104.08663* (2021).

[30] Pauli Virtanen, Ralf Gommers, Travis E Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, et al. 2020. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature methods* 17, 3 (2020), 261–272.

[31] Joe H Ward Jr. 1963. Hierarchical grouping to optimize an objective function. *Journal of the American statistical association* 58, 301 (1963), 236–244.

[32] Andrew Yates, Rodrigo Nogueira, and Jimmy Lin. 2021. Pretrained transformers for text ranking: BERT and beyond. In *Proceedings of the 14th ACM International Conference on web search and data mining*. 1154–1156.

[33] Jingtao Zhan, Jiaxin Mao, Yiqun Liu, Jiafeng Guo, Min Zhang, and Shaoping Ma. 2021. Jointly optimizing query encoder and product quantization to improve retrieval performance. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*. 2487–2496.

[34] Jingtao Zhan, Xiaohui Xie, Jiaxin Mao, Yiqun Liu, Jiafeng Guo, Min Zhang, and Shaoping Ma. 2022. Evaluating interpolation and extrapolation performance of neural retrieval models. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*. 2486–2496.

[35] Xinyu Zhang, Nandan Thakur, Odunayo Ogundepo, Ehsan Kamalloo, David Alfonso Hermelo, Xiaoguang Li, Qun Liu, Mehdi Rezagholizadeh, and Jimmy Lin. 2023. MIRACL: A Multilingual Retrieval Dataset Covering 18 Diverse Languages. *Transactions of the Association for Computational Linguistics* 11 (2023), 1114–1131.

[36] Justin Zobel and Alistair Moffat. 2006. Inverted files for text search engines. *ACM computing surveys (CSUR)* 38, 2 (2006), 6–es.

A RETRIEVAL SETTING

The entire retrieval pipeline uses the same approach as the standard ColBERTv2 [27] + PLAID [26], including the maxSim scoring function [11]. For querying the PLAID index, we use the best-performing query hyperparameters reported in a recent reproduction study of PLAID [17]: *nprobe*=8, t_{cs}=0.3 and *ndocs*=8192.

For the HNSW index, we construct it using generous construction hyperparameters $M = 12$ and $EF_{construction} = 200$, designed to optimise retrieval performance. At querying time, we use large k values at candidate generation time to ensure the performance is not impacted by the use of approximate search. These hyperparameter choices effectively mean our results are similar to non-approximate searches.

Otherwise, we use the default ColBERTv2 and JacolBERTv2 parameters, with a respective document length of 256 and 300 tokens. We do not perform any particular optimisation step to maximise absolute retrieval performance, as we are focused on the relative performance of different methods. The exact same settings are used for both the baseline runs, and all token pooling methods compared to them.